

Chapter

7 Distributed Databases and Client-Server Architectures

Chapter 7 Outline

1. Distributed Database Concepts
2. Data Fragmentation, Replication and Allocation
3. Types of Distributed Database Systems
4. Query Processing
5. Concurrency Control and Recovery
6. 3-Tier Client-Server Architecture

Distributed Database Concepts

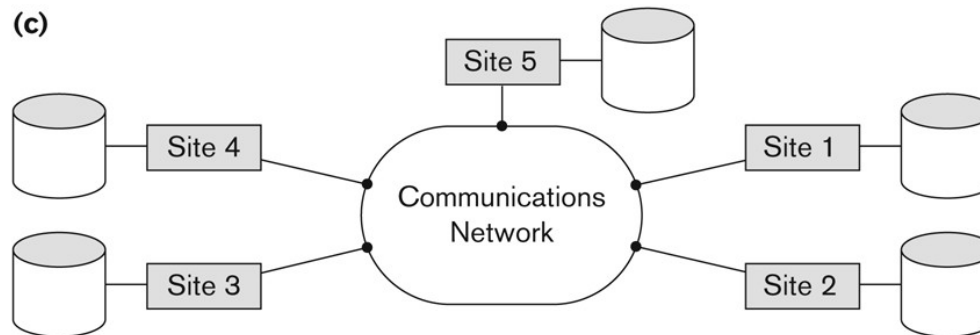
- A transaction can be executed by multiple networked computers in a unified manner.
- A **distributed database (DDB)** processes Unit of execution (a transaction) in a distributed manner. A distributed database (DDB) can be defined as
 - A distributed database (DDB) is a collection of multiple logically related database distributed over a computer network, and a distributed database management system as a software system that manages a distributed database while making the distribution transparent to the user.

Distributed Database System

- Advantages
 - Management of distributed data with different **levels of transparency**:
 - This refers to the physical placement of data (files, relations, etc.) which is not known to the user (distribution transparency).

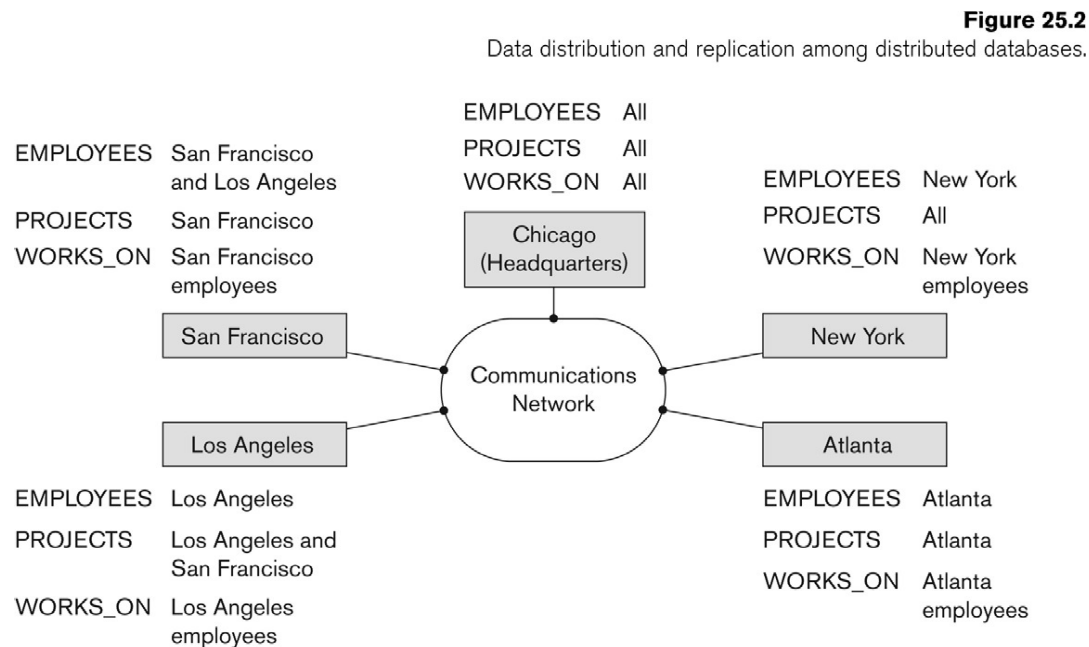
Figure 25.1

Some different database system architectures. (a) Shared nothing architecture. (b) A networked architecture with a centralized database at one of the sites. (c) A truly distributed database architecture.



Distributed Database System

- Advantages (transparency, contd.)
 - The EMPLOYEE, PROJECT, and WORKS_ON tables may be fragmented horizontally and stored with possible replication as shown below.



Distributed Database System

- Advantages (transparency, contd.)
 - **Distribution and Network transparency:**
 - Users do not have to worry about operational details of the network.
 - There is Location transparency, which refers to freedom of issuing command from any location without affecting its working.
 - Then there is Naming transparency, which allows access to any names object (files, relations, etc.) from any location.

Distributed Database System

- Advantages (transparency, contd.)
 - **Replication transparency:**
 - It allows to store copies of a data at multiple sites as shown in the above diagram.
 - This is done to minimize access time to the required data.
 - **Fragmentation transparency:**
 - Allows to fragment a relation horizontally (create a subset of tuples of a relation) or vertically (create a subset of columns of a relation).

Distributed Database System

- Other Advantages

- **Increased reliability and availability:**

- Reliability refers to system live time, that is, system is running efficiently most of the time.
 - Availability is the probability that the system is continuously available (usable or accessible) during a time interval.
 - A distributed database system has multiple nodes (computers) and if one fails then others are available to do the job.

Distributed Database System

- Other Advantages (contd.)
 - **Improved performance:**
 - A distributed DBMS fragments the database to keep data closer to where it is needed most.
 - This reduces data management (access and modification) time significantly.
 - **Easier expansion (scalability):**
 - Allows new nodes (computers) to be added anytime without changing the entire configuration.

Data Fragmentation, Replication and Allocation

- **Data Fragmentation**

- Split a relation into logically related and correct parts. A relation can be fragmented in two ways:
 - **Horizontal Fragmentation**
 - **Vertical Fragmentation**

Data Fragmentation, Replication and Allocation

■ Horizontal fragmentation

- It is a horizontal subset of a relation which contain those of tuples which satisfy selection conditions.
- Consider the Employee relation with selection condition ($DNO = 5$). All tuples satisfy this condition will create a subset which will be a horizontal fragment of Employee relation.
- A selection condition may be composed of several conditions connected by AND or OR.
- Derived horizontal fragmentation: It is the partitioning of a primary relation to other secondary relations which are related with Foreign keys.

Data Fragmentation, Replication and Allocation

■ Vertical fragmentation

- It is a subset of a relation which is created by a subset of columns. Thus a vertical fragment of a relation will contain values of selected columns. There is no selection condition used in vertical fragmentation.
- Consider the Employee relation. A vertical fragment of can be created by keeping the values of Name, Bdate, Sex, and Address.
- Because there is no condition for creating a vertical fragment, each fragment must include the primary key attribute of the parent relation Employee. In this way all vertical fragments of a relation are connected.

Data Fragmentation, Replication and Allocation

■ Data Replication

- Database is replicated to all sites.
- In full replication the entire database is replicated and in partial replication some selected part is replicated to some of the sites.
- Data replication is achieved through a replication schema.

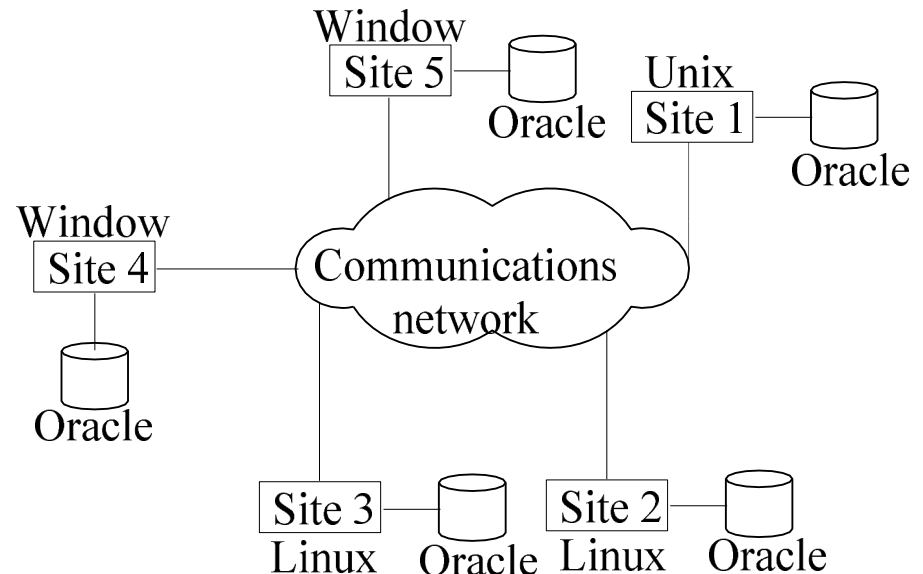
■ Data Distribution (Data Allocation)

- This is relevant only in the case of partial replication or partition.
- The selected portion of the database is distributed to the database sites.

Types of Distributed Database Systems

■ Homogeneous

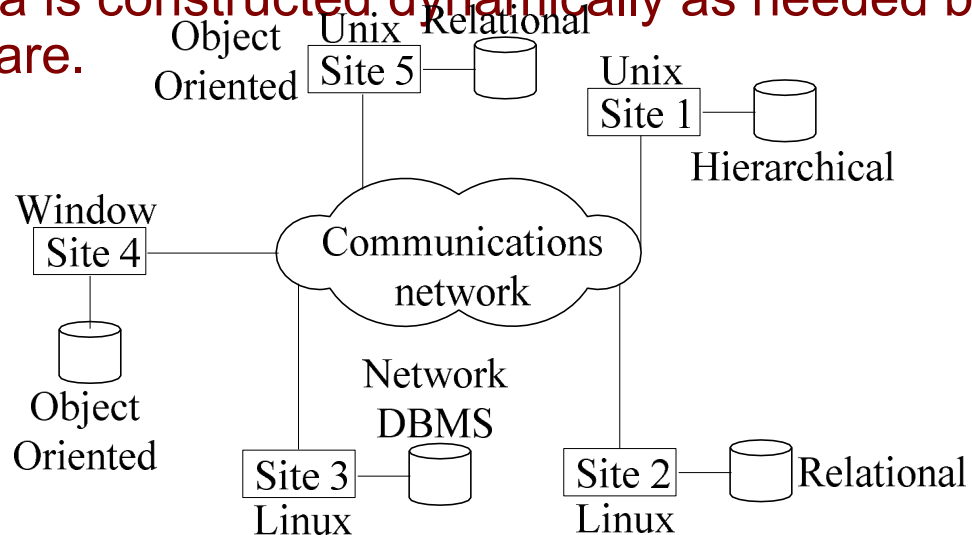
- All sites of the database system have identical setup, i.e., same database system software.
- The underlying operating system may be different.
 - For example, all sites run Oracle or DB2, or Sybase or some other database system.
- The underlying operating systems can be a mixture of Linux, Window, Unix, etc.



Types of Distributed Database Systems

- Heterogeneous

- Federated: Each site may run different database system but the data access is managed through a single conceptual schema.
 - This implies that the degree of local autonomy is minimum. Each site must adhere to a centralized access policy. There may be a global schema.
- Multidatabase: There is no one conceptual global schema. For data access a schema is constructed dynamically as needed by the application software.



Query Processing in Distributed Databases

■ Issues

■ Cost of transferring data (files and results) over the network.

- This cost is usually high so some optimization is necessary.
- Example relations: Employee at site 1 and Department at Site 2

- Employee at site 1. 10,000 rows. Row size = 100 bytes = 10^6 bytes.

Fname	Minit	Lname	<u>SSN</u>	Bdate	Address	Sex	Salary	Superssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	----------	-----

- Department at Site 2. 100 rows. Row size = 35 bytes = 3,500 bytes.

Dname	Dnumber	Mgrssn	Mgrstartdate
-------	---------	--------	--------------

- Q: For each employee, retrieve employee name and department name Where the employee works.

- Q: $\Pi_{Fname, Lname, Dname} (Employee \bowtie_{Dno = Dnumber} Department)$

Query Processing in Distributed Databases

■ Result

- The result of this query will have 10,000 tuples, assuming that every employee is related to a department.
- Suppose each result tuple is 40 bytes long. The query is submitted at site 3 and the result is sent to this site.
- Problem: Employee and Department relations are not present at site 3.

Query Processing in Distributed Databases

- Strategies:

1. Transfer Employee and Department to site 3.

- Total transfer bytes = $1,000,000 + 3500 = 1,003,500$ bytes.

2. Transfer Employee to site 2, execute join at site 2 and send the result to site 3.

- Query result size = $40 * 10,000 = 400,000$ bytes
Total transfer size = $400,000 + 1,000,000 = 1,400,000$ bytes.

3. Transfer Department relation to site 1, execute the join at site 1, and send the result to site 3.

- Total bytes transferred = $400,000 + 3500 = 403,500$ bytes.

- Optimization criteria: minimizing data transfer.

Query Processing in Distributed Databases

- Strategies:

1. Transfer Employee and Department to site 3.

- Total transfer bytes = $1,000,000 + 3500 = 1,003,500$ bytes.

2. Transfer Employee to site 2, execute join at site 2 and send the result to site 3.

- Query result size = $40 * 10,000 = 400,000$ bytes
Total transfer size = $400,000 + 1,000,000 = 1,400,000$ bytes.

3. Transfer Department relation to site 1, execute the join at site 1, and send the result to site 3.

- Total bytes transferred = $400,000 + 3500 = 403,500$ bytes.

- Optimization criteria: minimizing data transfer.

- Preferred approach: strategy 3.

Query Processing in Distributed Databases

- Consider the query
 - Q': For each department, retrieve the department name and the name of the department manager
- Relational Algebra expression:
 - $\Pi_{Fname, Lname, Dname} (Employee \bowtie_{Mgrssn = SSN} Department)$

Query Processing in Distributed Databases

- The result of this query will have 100 tuples, assuming that every department has a manager, the execution strategies are:
 1. Transfer Employee and Department to the result site and perform the join at site 3.
 - Total bytes transferred = $1,000,000 + 3500 = 1,003,500$ bytes.
 2. Transfer Employee to site 2, execute join at site 2 and send the result to site 3. Query result size = $40 * 100 = 4000$ bytes.
 - Total transfer size = $4000 + 1,000,000 = 1,004,000$ bytes.
 3. Transfer Department relation to site 1, execute join at site 1 and send the result to site 3.
 - Total transfer size = $4000 + 3500 = 7500$ bytes.

Query Processing in Distributed Databases

- The result of this query will have 100 tuples, assuming that every department has a manager, the execution strategies are:
 1. Transfer Employee and Department to the result site and perform the join at site 3.
 - Total bytes transferred = $1,000,000 + 3500 = 1,003,500$ bytes.
 2. Transfer Employee to site 2, execute join at site 2 and send the result to site 3. Query result size = $40 * 100 = 4000$ bytes.
 - Total transfer size = $4000 + 1,000,000 = 1,004,000$ bytes.
 3. Transfer Department relation to site 1, execute join at site 1 and send the result to site 3.
 - Total transfer size = $4000 + 3500 = 7500$ bytes.
- Preferred strategy: Choose strategy 3.

Query Processing in Distributed Databases

- Now suppose the result site is 2. Possible strategies :
 1. Transfer Employee relation to site 2, execute the query and present the result to the user at site 2.
 - Total transfer size = 1,000,000 bytes for both queries Q and Q'.
 2. Transfer Department relation to site 1, execute join at site 1 and send the result back to site 2.
 - Total transfer size for Q = $400,000 + 3500 = 403,500$ bytes and
 - for Q' = $4000 + 3500 = 7500$ bytes.

Query Processing in Distributed Databases

- **Semijoin:**
 - Objective is to reduce the number of tuples in a relation before transferring it to another site.
- **Example execution of Q or Q':**
 1. Project the join attributes of Department at site 2, and transfer them to site 1. For Q, $4 * 100 = 400$ bytes are transferred and for Q', $9 * 100 = 900$ bytes are transferred.
 2. Join the transferred file with the Employee relation at site 1, and transfer the required attributes from the resulting file to site 2. For Q, $34 * 10,000 = 340,000$ bytes are transferred and for Q', $39 * 100 = 3900$ bytes are transferred.
 3. Execute the query by joining the transferred file with Department and present the result to the user at site 2.

Concurrency Control and Recovery

- Distributed Databases encounter a number of concurrency control and recovery problems which are not present in centralized databases. Some of them are listed below.
 - Dealing with multiple copies of data items
 - Failure of individual sites
 - Communication link failure
 - Distributed commit
 - Distributed deadlock

Concurrency Control and Recovery

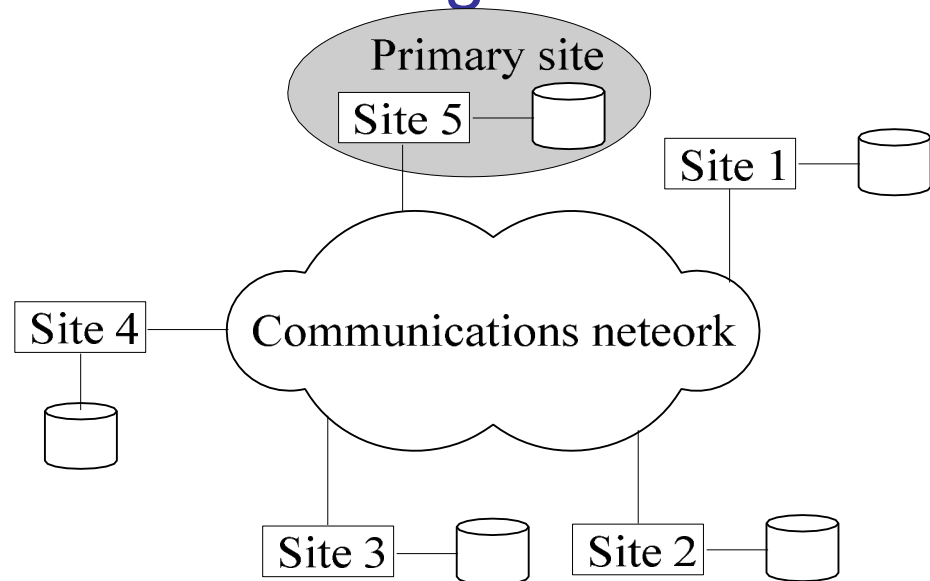
- Details
 - Dealing with multiple copies of data items:
 - The concurrency control must maintain global consistency. Likewise the recovery mechanism must recover all copies and maintain consistency after recovery.
 - Failure of individual sites:
 - Database availability must not be affected due to the failure of one or two sites and the recovery scheme must recover them before they are available for use.

Concurrency Control and Recovery

- Details (contd.)
 - Communication link failure:
 - This failure may create network partition which would affect database availability even though all database sites may be running.
 - Distributed commit:
 - A transaction may be fragmented and they may be executed by a number of sites. This requires a two or three-phase commit approach for transaction commit.
- Distributed deadlock:
 - Since transactions are processed at multiple sites, two or more sites may get involved in deadlock. This must be resolved in a distributed manner.

Concurrency Control and Recovery

- Distributed Concurrency control based on a distributed copy of a data item
- **Primary site technique:** A single site is designated as a primary site which serves as a coordinator for transaction management.



Concurrency Control and Recovery

- Transaction management:
 - Concurrency control and commit are managed by this site.
 - In two phase locking, this site manages locking and releasing data items. If all transactions follow two-phase policy at all sites, then serializability is guaranteed.

Concurrency Control and Recovery

- Transaction Management

- Advantages:

- An extension to the centralized two phase locking so implementation and management is simple.
 - Data items are locked only at one site but they can be accessed at any site.

- Disadvantages:

- All transaction management activities go to primary site which is likely to overload the site.
 - If the primary site fails, the entire system is inaccessible.
 - To aid recovery a backup site is designated which behaves as a shadow of primary site. In case of primary site failure, backup site can act as primary site.

Concurrency Control and Recovery

■ **Primary Copy Technique:**

- In this approach, instead of a site, a data item partition is designated as primary copy. To lock a data item just the primary copy of the data item is locked.
- Advantages:
 - Since primary copies are distributed at various sites, a single site is not overloaded with locking and unlocking requests.
- Disadvantages:
 - Identification of a primary copy is complex. A distributed directory must be maintained, possibly at all sites.

Concurrency Control and Recovery

■ Recovery from a coordinator failure

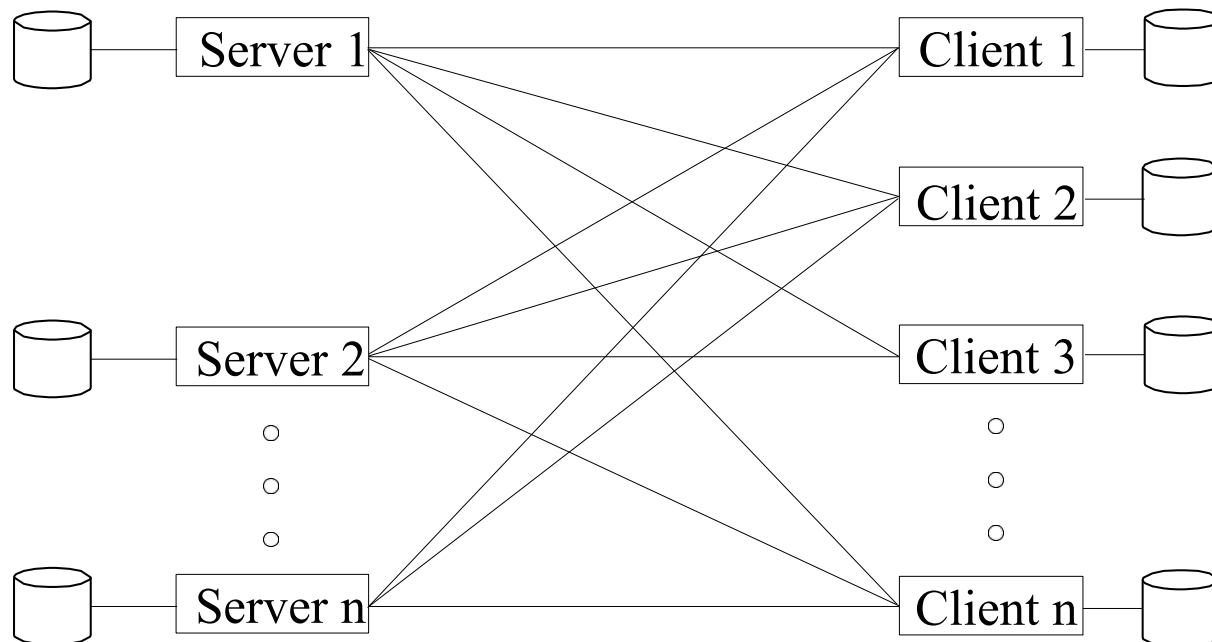
- In both approaches a coordinator site or copy may become unavailable. This will require the selection of a new coordinator.
- Primary site approach with no backup site:
 - Aborts and restarts all active transactions at all sites. Elects a new coordinator and initiates transaction processing.
- Primary site approach with backup site:
 - Suspends all active transactions, designates the backup site as the primary site and identifies a new back up site. Primary site receives all transaction management information to resume processing.
- Primary and backup sites fail or no backup site:
 - Use election process to select a new coordinator site.

Concurrency Control and Recovery

- Concurrency control based on voting:
 - There is no primary copy of coordinator.
 - Send lock request to sites that have data item.
 - If majority of sites grant lock then the requesting transaction gets the data item.
 - Locking information (grant or denied) is sent to all these sites.
 - To avoid unacceptably long wait, a time-out period is defined. If the requesting transaction does not get any vote information then the transaction is aborted.

Client-Server Database Architecture

- It consists of clients running client software, a set of servers which provide all database functionalities and a reliable communication infrastructure.



Client-Server Database Architecture

- Clients reach server for desired service, but server does reach clients.
- The server software is responsible for local data management at a site, much like centralized DBMS software.
- The client software is responsible for most of the distribution function.
- The communication software manages communication among clients and servers.

Client-Server Database Architecture

- The processing of a SQL queries goes as follows:
 - Client parses a user query and decomposes it into a number of independent sub-queries. Each subquery is sent to appropriate site for execution.
 - Each server processes its query and sends the result to the client.
 - The client combines the results of subqueries and produces the final result.

Recap

- Distributed Database Concepts
- Data Fragmentation, Replication and Allocation
- Types of Distributed Database Systems
- Query Processing
- Concurrency Control and Recovery
- 3-Tier Client-Server Architecture

*The end of the
course !!!
Thank you.*

